

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 2

PHP-Grundlagen Teil 1

Nachdem in Vorlesung 1 die grundlegende Funktionsweise und der Weg eines Webrequests dargestellt wurden, beginnt nun der eigentliche Inhalt dieser Lehrveranstaltung: serverseitige Webentwicklung mit der Skriptsprache PHP.

2.1 Allgemeines zu PHP

PHP – ein rekursives Akronym für *PHP: Hypertext Preprocessor* – ist eine serverseitige Skriptsprache, die vornehmlich zur Erzeugung von HTML-Inhalten verwendet wird. Die Sprache ist (wie JavaScript) interpretiert, es wird also der Code zur Laufzeit eingelesen und verarbeitet.

PHP ist ein Open-Source-Projekt der PHP Group, die zentrale Webseite lautet <https://www.php.net/>.

2.1.1 Geschichte

PHP wurde vom dänischen Programmierer Rasmus Lerdorf entwickelt. Er stellte im Juni 1995 seine *Personal Homepage Tools* (PHP Tools) vor. Sie waren eine Reihe von in der Programmiersprache C geschriebenen Funktionen, mit denen er Zugriffe auf seinen Online-Lebenslauf protokollieren konnte.

Im Zuge seiner Arbeit entwickelte Lerdorf diese Tools weiter und veröffentlichte im April 1996 *PHP/FI*. FI stand hierbei für Form Interpreter. Diese Version kann schon als eigene, wenn auch noch begrenzte Skriptsprache angesehen werden, da bereits ein eigener Parser die verschiedenen Befehle interpretierte.

Im Juni 1998 erschien *PHP3*. Das Projekt war mittlerweile von einer One-Man-Show zu einem kleinen Open-Source-Projekt gewachsen. Der Parser wurde von Grund auf neu geschrieben, zusätzliche Funktionen wie Datenbankverbindung trugen zu einer größeren Verbreitung von PHP bei.

Im Jahr 2000 erschien *PHP4*. Mittlerweile hatten die Entwickler des Parsers (Andi Gutmans und Zeev Suraski) eine eigene Firma – Zend Technologies – gegründet und schufen mit der Zend Engine 1 den Kern der PHP-Implementierung. Durch das enorme Wachstum des World Wide Webs Ende der 1990er Jahre gewann auch PHP stark an Bedeutung. Durch seine einfache Erlernbarkeit etablierte es sich zu dieser Zeit als die dominante Skriptsprache im Web.

2004 erschien dann die lange erwartete *Version 5* von PHP. Kern war die Zend Engine II, die nun erstmals auch objektorientierte Entwicklung in PHP unterstützte. Weitere Konzepte wie Exceptions, Integration weiterer Datenbank Systeme oder verschiedener Erweiterungen wie XML-Parsern machten diese Version zu einer der wichtigsten im PHP Lebenszyklus.

Die *Version 5.1* (vom November 2005) führte vor allem Leistungsverbesserungen ein, in *Version 5.2* (November 2006) wurde die Speicherverwaltung verbessert, der Umgang mit dem Datenformat JSON hinzugefügt sowie neue Methoden für die Zeit- und Datumsverwaltung eingeführt.

Version 5.3 (Ende Juni 2009) brachte die Einführung von Namespaces, verbesserte XML-Unterstützung und noch einige weitere Neuerungen. In *PHP 5.4* vom März 2012 wurden Traits (wiederverwendbare Methodensammlungen), eine Kurzschreibweise für Arrays sowie weitere Sprachverbesserungen und Sicherheitsfeatures eingeführt.

PHP 5.5 (Juni 2013) führte die Unterstützung von Generatoren und eine Verbesserung der Ausnahmebehandlung ein. *PHP Version 5.6* wurde im August 2014 veröffentlicht. Neu waren konstante skalare Ausdrücke, variadische Funktionen oder auch automatisches Argument-Unpacking.

Anfang Dezember 2015 erschien *PHP 7*. Dabei wurde sehr viel der zugrundeliegenden Zend Engine neu geschrieben, was vor allem Performancegewinn brachte. Ebenso wurden viele alte Funktionalitäten, die aus Kompatibilitätsgründen in Version 5 enthalten waren, entfernt. Ebenso gibt es viele neue Sprachfunktionen wie etwa skalare Typdeklarationen, Deklarationen für Rückgabewerte, neue Operatoren, anonyme Klassen und noch einiges mehr¹.

Im Dezember 2016 wurde *PHP 7.1* veröffentlicht. Neue Funktionen umfassen sogenannte „Nullable Types“, den Rückgabotyp `void`, mehrere Sichtbarkeiten für Klassenkonstanten und noch weitere Dinge².

PHP 7.2 wurde Ende November 2017 vorgestellt. Hierbei gab es nur kleine Änderungen, wie etwa die Möglichkeit, `object` als Rückgabotyp oder Typdeklaration anzugeben, die Möglichkeit, abstrakte Methoden zu überschreiben, ein neuer Algorithmus für die Passwort-Hashing API, sowie einige weitere Änderungen³.

PHP 7.3 wurde im Dezember 2018 vorgestellt. Hier kamen Änderungen für die Heredoc- und Nowdoc-Syntax sowie einige Änderungen bei Stringfunktionen, die die Übergabeparameter jetzt strenger prüfen⁴.

PHP 7.4 erschien Ende November 2019. Eine große und wichtige Neuerung waren sogenannte „typed properties“, also die Möglichkeit, Datentypen bei Klasseneigenschaften zu spezifizieren. Ebenso wurden Pfeilfunktionen (arrow functions), der Null coalescing Zuweisungsoperator und noch einige weitere Dinge hinzugefügt⁵.

PHP 8.0 wurde am 20. November 2020 veröffentlicht. Die Version brachte einige Neuigkeiten mit, die zum Teil mit alten Versionen nicht mehr kompatibel sind. Wichtige Änderungen waren unter anderem eine „just-in-time Kompilation“, das Statement `match`, Union-Types, benannte Argumente, Constructor-Property-Promotion oder der

¹<https://www.php.net/manual/de/migration70.new-features.php>

²<https://www.php.net/manual/de/migration71.new-features.php>

³<https://www.php.net/manual/de/migration72.new-features.php>

⁴<https://www.php.net/manual/de/migration73.new-features.php>

⁵<https://www.php.net/manual/de/migration74.new-features.php>

Nullsafe-Operator⁶.

PHP 8.1 erschien am 25. November 2021. Neu waren unter anderem der Enumeration-Datentyp, der **never** Rückgabetypp für Funktionen, die nie zurückkehren, finale Klassenkonstanten oder **readonly** Eigenschaften⁷.

PHP 8.2 wurde am 8. Dezember 2022 vorgestellt. Es wurden **readonly**-Klassen eingeführt, Verbesserungen am Typensystem vorgenommen (Rückgabewerte **null** und **true**), Konstanten in Traits sowie ein neuer, zuverlässiger Zufallszahlengenerator namens **Randomizer** vorgestellt⁸.

PHP 8.3 ist am 23. November 2023 erschienen. Neuerungen waren unter anderem typisierte Klassenkonstanten, anonyme **readonly**-Klassen, eine Funktion zur JSON-Validierung, sowie Verbesserungen am **Randomizer**⁹.

PHP 8.4 wurde am 21. November 2024 veröffentlicht. Neu waren Property Hooks und asymmetrische Sichtbarkeit in Klassen, die Getter- und Setter-Methoden vereinfachen bzw. reduzieren, Updates in den DOM-Klassen und Lazy Objects¹⁰.

Die aktuellste Version ist *PHP 8.5*, welches am 20. November 2025 veröffentlicht wurde. Zentrale Neuerung ist der Pipe-Operator (**|>**), der das Ergebnis des linken Ausdrucks als Argument an den Funktionsaufruf auf der rechten Seite, um verschachtelte Aufrufe in eine linear lesbare Kette zu transformieren. Weiters neue Möglichkeiten zum Clonen von Objekten, Verbesserungen für Closures und Methoden zum URI-Parsing¹¹. Die aktuelle Release-Version ist 8.5.5 vom 9. April 2026.

Eine Version 6 von PHP gab es übrigens nie. Zentraler Kern war die komplette Neuimplementierung unter Berücksichtigung von Unicode, was sich jedoch aus vielfältigen Gründen als problematisch herausstellte und daher verworfen wurde. Einige geplante Neuerungen flossen bereits ab PHP 5.3 ein, der flächendeckende Unicode-Support wurde jedoch nicht implementiert.

2.1.2 Alternativen

PHP ist nicht die einzige Möglichkeit, serverseitig HTML-Code zu erzeugen. Die folgenden Technologien existieren ebenfalls – die Liste ist jedoch bei weitem nicht vollständig.

JavaScript

JavaScript wurde ursprünglich als clientseitige Sprache entworfen (wie sie auch in Web Fundamentals verwendet wurde), durch die Entwicklung von *Node.js*¹² wurde jedoch auch der serverseitige Einsatz populär. Mit seiner nicht-blockierenden Event-Loop-Architektur bietet Node.js eine leistungsstarke Umgebung für skalierbare Netzwerkanwendungen. Ein bekanntes Framework ist etwa *Express.js*¹³. Node.js ist Thema in Web Architectures (WAR3VO).

⁶<https://www.php.net/manual/de/migration80.new-features.php>

⁷<https://www.php.net/manual/de/migration81.new-features.php>

⁸<https://www.php.net/manual/de/migration82.new-features.php>

⁹<https://www.php.net/manual/de/migration83.new-features.php>

¹⁰<https://www.php.net/manual/de/migration84.new-features.php>

¹¹<https://www.php.net/manual/en/migration85.new-features.php>

¹²<https://nodejs.org/>

¹³<https://expressjs.com/>

Python

*Python*¹⁴ ist eine universell einsetzbare, höhere Programmiersprache die objektorientiert oder auch funktional eingesetzt werden kann. Im Web ist sie die Grundlage bekannter Webframeworks wie etwa Django¹⁵ oder Flask¹⁶ und wird zumeist auch mit diesen verwendet. Darüber hinaus ist die Sprache in der wissenschaftlichen Community für mathematische Berechnungen oder Machine-Learning beliebt und wird auch im MTD-Studium unterrichtet.

Java

Die Programmiersprache *Java*¹⁷ ist eine weit verbreitete, objektorientierte Programmiersprache, die für ihre Portabilität, Sicherheit und Robustheit bekannt ist und sehr oft in komplexen Projekten zum Einsatz kommt. Es existieren eine Vielzahl an speziellen *Java Web-Frameworks*, mit denen sich dynamisch Inhalte erzeugen lassen. Beispiele sind etwa Spring¹⁸ oder Struts¹⁹. Diese Themen werden ausführlich im Masterstudium Interactive Media behandelt.

Scala

*Scala*²⁰ ist eine moderne Programmiersprache, die Konzepte der objektorientierten und der funktionalen Programmierung vereint. Sie läuft auf der Java Virtual Machine (JVM) und zeichnet sich durch ein starkes, statisches Typsystem sowie eine prägnante Syntax aus. Im Web-Bereich wird Scala häufig für hochperformante, skalierbare Backends und Systeme mit hoher Nebenläufigkeit eingesetzt. Ein bekanntes Framework zur Erzeugung dynamischer Web-Inhalte ist das *Play Framework*²¹. Durch die volle Interoperabilität mit Java können bestehende Java-Bibliotheken nahtlos eingebunden werden, was Scala besonders für komplexe Enterprise-Anwendungen und im Bereich Big Data attraktiv macht.

Ruby

*Ruby*²² ist eine primär objektorientierte Programmiersprache, die jedoch auch prozedural und funktional eingesetzt werden kann. Im Web wird sie meist zusammen mit Webframeworks oder als domänenspezifische Sprache verwendet. Bekannte Vertreter sind etwa Ruby on Rails²³ oder Sinatra²⁴. Die Sprache hat sich nebenbei auch als Skriptsprache etabliert, die serverseitige Aufgaben (z. B. als Ersatz für Bash-Skripte) ausführt.

¹⁴<https://www.python.org/>

¹⁵<https://www.djangoproject.com/>

¹⁶<https://palletsprojects.com/projects/flask/>

¹⁷<https://www.oracle.com/java/>

¹⁸<https://spring.io/>

¹⁹<https://struts.apache.org/>

²⁰<https://www.scala-lang.org/>

²¹<https://www.playframework.com/>

²²<https://www.ruby-lang.org/>

²³<https://rubyonrails.org/>

²⁴<https://sinatrarb.com/>

Ebenso sind bekannte Softwarepakete, wie etwa der statische Seitengenerator Jekyll²⁵, in Ruby geschrieben.

2.1.3 Verbreitung und Beispiele

Die Seite W3Techs führt eine Analyse der Verbreitung von serverseitigen Programmiersprachen durch [7]. Mitte April 2026 ergibt sich dabei folgende Verteilung:

- PHP – 71,7 % (2025: 74,7 %, 2024: 76,5 %, 2023: 77,6 %),
- Ruby – 6,7 % (2025: 6,2 %, 2024: 5,7 %, 2023: 5,2 %),
- JavaScript – 6,0 % (2025: 4,2 %, 2024: 3,2 %, 2023: 2,3 %),
- Java – 5,4 % (2025: 5,1 %, 2024: 4,7 %, 2023: 4,7 %),
- Scala – 5,0 % (2025: 4,1 %, 2024: 3,0 %, 2023: 2,8 %).

Dabei verwenden derzeit 58,2 % (2025: 41,9 %, 2024: 25,3 %, 2023: 9,6 %) aller PHP-Seiten PHP 8, 33,0 % (2025: 46,1 %, 2024: 58,0 %, 2023: 67,8 %) PHP 7 und 8,7 % (2025: 11,8 %, 2024: 16,6 %, 2023: 22,4 %) PHP 5. Ältere Versionen spielen praktisch keine Rolle mehr.

Folgende bekannte Seiten sind mit PHP erstellt:

- Facebook (<https://www.facebook.com/>, mit Hack²⁶ als „Dialekt“),
- Wikipedia (<https://www.wikipedia.org/>),
- Flickr (<https://flickr.com/>),
- Etsy (<https://www.etsy.com/>),
- Wordpress (<https://wordpress.com/>).

2.2 Serverseitige Webentwicklung

Während sich die Vorlesung zu Web Fundamentals lediglich mit der Erstellung von statischen Webseiten beschäftigte, sind mit PHP nun dynamische Seiten möglich.

2.2.1 Wozu überhaupt?

Statische Seiten eignen sich gut für gleichbleibende Inhalte. Diese werden einmal in HTML verfasst und gegebenenfalls gelegentlich angepasst. Dazu muss jedes Mal das HTML-Markup händisch geändert bzw. neu erzeugt werden. Ein automatisches Reagieren auf neue oder von Anwender*innen bereitgestellte Daten ist nicht möglich.

Dynamische Webseiten zielen genau darauf ab. Mit ihnen können Inhalte programmatisch erstellt werden, ohne diese genau kennen zu müssen. So kann etwa mittels eines Formulars der Benutzer*innenname abgefragt und auf allen Folgeseiten angezeigt werden. Für derartige Dinge reicht eine Markupsprache wie HTML also nicht mehr aus, es muss eine Skriptsprache zum Einsatz kommen. Nun könnte argumentiert werden, dass Client-side-JavaScript für solche Dinge durchaus geeignet sei. Dies stimmt jedoch nur begrenzt, denn JavaScript dient zum Verändern einer bereits bestehenden Seite, bietet aber (clientseitig) bei weitem nicht alle Funktionalitäten für dynamische Webseiten.

²⁵<https://jekyllrb.com/>

²⁶[https://de.wikipedia.org/wiki/Hack_\(Programmiersprache\)](https://de.wikipedia.org/wiki/Hack_(Programmiersprache))

Sollen etwa Daten aus einer Datenbank eingelesen und auf der Seite dargestellt werden, ist JavaScript im Browser allein nicht mehr geeignet.

Hier ist eine serverseitige Skriptsprache wie PHP nötig, die all diese Dinge erledigt, *bevor* die Seite an den Browser geschickt und angezeigt wird. Dies geschieht letztendlich aber wieder in HTML, d. h. serverseitige Skriptsprachen wie PHP generieren HTML-Markup, das an den Browser gesendet wird. Spätere Vorlesungen demonstrieren auch noch andere Anwendungszwecke wie REST APIs, die nicht HTML generieren, sondern Daten (z. B. im JSON-Format) liefern.

2.2.2 Funktionsweise

Während bei einer statischen Seite das HTML-Markup fix in der HTML-Datei festgeschrieben wird, können PHP-Dateien sowohl statische HTML-Inhalte als auch dynamische, mit PHP definierte Abschnitte enthalten. Da der Webbrowser jedoch nur HTML interpretieren und mit PHP-Code nichts anfangen kann (außer diesen vielleicht 1:1 als Text auszugeben), muss es eine Instanz geben, die diesen PHP-Code verarbeitet und ihn in entsprechendes HTML umwandelt. Diese Komponente ist die *PHP-Engine*, auch *PHP-Parser* oder *PHP-Interpreter* genannt. Sie ist Teil (ein sogenanntes Modul) eines Webserver (wie bereits in Abschnitt 1.4 erläutert). Somit können PHP-Seiten nicht einfach lokal aus dem Dateisystem geöffnet werden, sondern müssen immer von einem Webserver angefordert werden, da sonst die PHP-Engine ja nicht zur Verfügung steht.

Wird nun eine PHP-Seite im Browser angefordert (es soll etwa die Datei `file.php` geladen werden), so schickt der Browser einen Request an den Webserver, auf dem diese Seite liegt. Dies kann z. B. über den URL `https://www.myserver.com/file.php` (online im WWW), oder `http://localhost:8080/file.php` (lokal am eigenen Rechner, mehr dazu in Abschnitt 2.2.3) sein. Im Webserver wird nun die Ressource (`file.php`) aus dem Dateisystem angefordert. Aufgrund der Dateiendung `.php` weiß der Webserver, dass er die Datei nun der PHP-Engine übergeben muss (wäre eine HTML-Datei angefordert worden, hätte er diese direkt zurücksenden können).

Die PHP-Engine liest nun den Inhalt der PHP-Datei (PHP-Code, evtl. auch statische HTML-Teile) ein und interpretiert diese. Im Rahmen dieser Verarbeitung erzeugt der enthaltene PHP-Code (in der Regel) HTML-Output, der im Anschluss zusammen mit den ebenfalls in der Datei enthaltenen HTML-Teilen als Response zurück an den Browser geschickt und dort angezeigt wird. Dies entspricht Schritt 8 in Abbildung 1.7.

Da es sich bei PHP um eine Skriptsprache handelt, kann der HTML-Output algorithmisch (mit Hilfe von Schleifen, Bedingungen etc.) erzeugt werden, darum wird auch von dynamischen Webseiten gesprochen.

Abbildung 2.1 stellt in Vorlesung 1, Abschnitt 1.4 gezeigten Kommunikationsablauf, erweitert um die PHP-Engine, grafisch dar.

2.2.3 Lokaler Webserver mit Docker

Es wird also zur Erzeugung von Webseiten mittels PHP ein Computer mit Webserversoftware und installiertem PHP-Modul benötigt. Sollen Datenbanken verwendet werden, muss auch eine solche installiert und konfiguriert sein. Während diese Einzelkomponenten bei der Zusammenstellung eines sogenannten Produktivwebserver, also eines Server, der für den Einsatz im Internet verwendet wird, in der Regel von einem Hoster

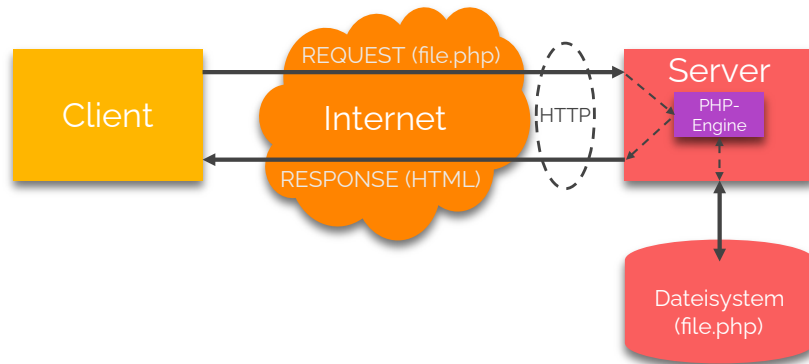


Abbildung 2.1: Client-Server Kommunikation über das Internet. Die PHP-Engine im Webserver kümmert sich dabei um das Verarbeiten des PHP-Codes und das Erzeugen der HTML-Ausgabe.

konfiguriert wird, gibt es für die lokale Webentwicklung spezielle Lösungen.

Bei *Docker* handelt es sich um eine sogenannte Containervirtualisierung. Hier wird nicht ein gesamtes Betriebssystem virtuell zur Verfügung gestellt (wie etwa bei einer virtuellen Maschine), viel mehr laufen Anwendungen eingeschränkt in Containern. Diese erhalten nur die notwendigen Ressourcen, um die darin enthaltenen Programme auszuführen.

Container sind in sogenannten Images definiert. Aus ihnen werden ein oder mehrere Container gestartet. Der Container ist dann die aktive Instanz in der gearbeitet wird. Container verwenden sehr oft Volumes, die das Dateisystem repräsentieren und im umgebenden Host-System gespeichert werden. Somit liegen die Dateien sicher und unabhängig von den Containern im Host-Betriebssystem und bleiben auch beim Zerstören der Container erhalten.

Das Projekt *fhoee-web-dock*²⁷ ist eine Sammlung von Docker-Containern, die für die Webentwicklung an der FH Oberösterreich erstellt wurden. Sie enthalten einen Webserver mit PHP, einen Container mit dem Datenbanksystem MariaDB sowie das Administrationstool phpMyAdmin.

Mit dem Befehl `docker compose up -d` können die Container erstmalig erstellt und gestartet werden. In weiterer Folge können diese mittels `docker compose stop` gestoppt und mit `docker compose start` wieder gestartet werden. `docker compose down` zerstört die Container komplett. Da die *fhoee-web-dock*-Images auf den offiziellen Docker-Images für PHP, MariaDB und phpMyAdmin beruhen, wird beim Erstellen des Containers immer die neueste Version verwendet. Über das `CleanReinstall.bat` bzw. `CleanReinstall.sh` können die Container gestoppt, die lokalen Images mit allen temporären Dateien entfernt, neu heruntergeladen, neu erstellt und wieder gestartet werden.

Laufende Container erstellen einen Webserver, der unter `http://localhost:8080` bzw. `https://localhost:7443` verfügbar ist. Damit die eigenen Webprojekte dort angezeigt werden, müssen sie in dem Ordner `webapp` erstellt werden. Eine Datei `webapp/file.php` ist somit im Browser unter `http://localhost:8080/file.php` verfügbar.

²⁷<https://github.com/Digital-Media/fhoee-web-dock>

Das Datenbank-Tool phpMyAdmin kann unter `http://localhost:8082` bzw. `https://localhost:8443` aufgerufen werden.

Auf die Datenbank kann aus dem PHP-Container über den Hostnamen `db` und den Port `3306` und vom eigenen Betriebssystem über den Host `localhost` und den Port `6033` zugegriffen werden.

2.3 PHP-Sprachgrundlagen

In den folgenden Abschnitten werden die Grundzüge der Skriptsprache PHP vorgestellt. PHP wird primär für die dynamische Generierung von Websites eingesetzt, weshalb mit der Einbettung von PHP-Code begonnen wird. Es folgen Abschnitte über die lexikale Struktur, Datentypen, Variablen, Operatoren und Kontrollstrukturen. Da die Rückgabe von Text (zum Erzeugen des HTML-Outputs) essentiell ist, schließt dieser Abschnitt mit der Ausgabe von Strings und wichtigen Stringfunktionen.

2.3.1 Einbettung in HTML

PHP-Code wird in der Regel in HTML-Markup eingebettet. Statische – also immer gleichbleibende Inhalte (wie z. B. der DOCTYPE oder auch Informationen im `<head>`) werden ganz normal in HTML verfasst. Alle veränderbaren Stellen, deren Ausgabe dynamisch sein soll und etwa von Eingaben der Benutzer*innen oder Abfragen aus einer Datenbank abhängen, werden über PHP generiert. Somit muss PHP-Code in HTML-Dateien (allerdings jetzt mit der Dateierweiterung `.php`) eingebettet werden, und zwar so, dass es für den PHP-Parser erkennbar ist, welche Bereiche er verarbeiten muss (den PHP-Code) und welche Bereiche er unverändert zurückgibt (den HTML-Code).

PHP Tags

Die am weitesten verbreitete Variante, PHP-Code zu kennzeichnen, ist mittels einer XML-konformen *Processing Instruction*, meist einfach *PHP Tags* genannt. Diese beginnt (wie jede Processing Instruction) mit einer spitzen Klammer und einem Fragezeichen, gefolgt vom Schlüsselwort `php`. Geschlossen wird die Instruktion mit einem Fragezeichen und der umgekehrten spitzen Klammer. Die folgende Zeile ist eine korrekte Einbettung von PHP-Code:

```
1 <?php echo "PHP"; ?>
```

Ein solcher „PHP-Block“ kann sich auch über mehrere Zeilen erstrecken:

```
1 <?php
2 echo "Erste Zeile";
3 echo "Zweite Zeile";
4 ?>
```

Das in beiden Beispielen gezeigte Sprachkonstrukt `echo` dient zur Ausgabe von Text. Mehr dazu in Abschnitt 2.3.2 auf Seite 2-10.

Das folgende Beispiel zeigt ein HTML-Grundgerüst, mit dem die Ausgabe im `<body>` mittels PHP erzeugt wird. Dies soll die bereits erwähnte Verzahnung/Einbettung von HTML und PHP verdeutlichen.

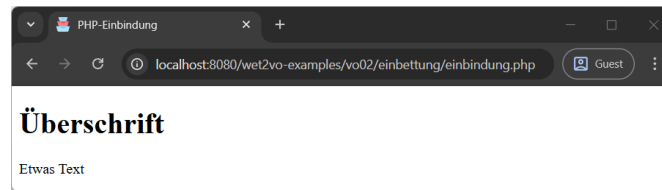


Abbildung 2.2: Die Ausgabe entspricht in diesem Fall der einer statischen HTML-Seite. Inhalte im `<body>` wurden aber vom PHP-Parser interpretiert und ausgegeben. Der URL `http://localhost:8080/wet2vo-examples/vo02/einbettung/einbindung.php` in der Adresszeile zeigt, dass die Datei über den Webserver (in diesem Fall über den *fhoee-web-dock* Docker Container) aufgerufen wurde.

```
1 <!DOCTYPE html>
2 <html lang="de">
3 <head>
4   <meta charset="utf-8">
5   <title>PHP-Einbindung</title>
6 </head>
7 <body>
8 <?php
9 echo "<h1>Überschrift</h1>";
10 echo "<p>Etwas Text</p>";
11 ?>
12 </body>
13 </html>
```

Der dadurch erzeugte Quellcode sieht wie folgt aus. Das Ergebnis im Browser ist in Abbildung 2.2 ersichtlich.

```
1 <!DOCTYPE html>
2 <html lang="de">
3 <head>
4   <meta charset="utf-8">
5   <title>PHP Einbindung</title>
6 </head>
7 <body>
8   <h1>Überschrift</h1>
9   <p>Etwas Text</p>
10 </body>
11 </html>
```

Achtung: Enthält eine Datei ausschließlich PHP-Code, sollte der schließende Tag entfallen um Probleme mit unerwünschten Whitespace-Zeichen am Ende zu verhindern:

```
1 <?php
2 echo "Hallo Welt";
3 // ... mehr Code
4 echo "Letzte Ausgabe";
5 // Das Script endet hier ohne Angabe von ?>
```

Short Open Tags

Neben dieser Variante existiert mit den *Short Open Tags* noch eine weitere. Dabei wird PHP-Code in eine Processing Instruction gesetzt, der Identifikator `php` entfällt

jedoch. Diese Variante ist zwar kürzer, ist aber in den meisten PHP-Konfigurationen aus Sicherheitsgründen (zu große Ähnlichkeit z. B. mit Processing Instructions) über die Einstellung `short_open_tag=false` in `php.ini` deaktiviert. Die Short Tag Syntax sieht wie folgt aus:

```
1 <? echo "PHP"; ?>
```

Short Echo Tags: Ein Spezialfall der Short Open Tags sind die *Short Echo Tags*. Sie sind für einzeilige Ausgabe von Text gedacht. Anstatt `<?php echo "Hallo Welt"; ?>` zu schreiben, kann ein Short Echo Tag wie folgt verwendet werden:

```
1 <?="Hallo Welt" ?>
```

Seit PHP 5.4 sind die Short Echo Tags immer verfügbar, auch wenn die Short Open Tags deaktiviert sind. Sie können daher bedenkenlos verwendet werden.

2.3.2 Ausgabe von Strings

Bevor mit weiteren Details zu PHP fortgefahren wird, ist es notwendig zu wissen, wie in PHP *Output* erzeugt werden kann. Denn schlussendlich sollen am Server mittels PHP ja HTML-Seiten erzeugt werden, die dann im Browser angezeigt werden. Auch die Beispiele in den folgenden Abschnitten geben oft Inhalt aus, um bestimmte Vorgänge im Browser testbar zu machen.

PHP kennt zwei Möglichkeiten, Text auszugeben:

- `echo` und
- `print()`.

Ausgabe mit echo

`echo`²⁸ ist keine Funktion, sondern ein sogenanntes Sprachkonstrukt (wie etwa auch `if`, `for`, `return` und ähnliche). Daher werden die Argumente (der Text), die `echo` zur Ausgabe übergeben werden, nicht geklammert. Die Syntax lautet:

```
1 echo $str;
```

`$str` ist eine Variable (siehe dazu Abschnitt 2.3.5), die Text (einen String) enthält. Folgende Beispiele zeigen die Ausgabe von Text mittels `echo`:

```
1 echo "Hallo Welt"; // Keine Variable, String wird direkt angegeben
2 echo "<h1>Überschrift</h1>"; // Ein HTML-Fragment wird ausgegeben
3 echo $text; // Inhalt der Variable $text wird ausgegeben
```

Mit den in Abschnitt 2.3.1 angesprochenen Short Echo Tags sieht die Syntax wie folgt aus:

```
1 <?=" $str ?>
```

Die drei Beispiele werden wie folgt damit geschrieben:

```
1 <?="Hallo Welt" ?>
2 <?="<h1>Überschrift</h1>" ?>
3 <?=" $text ?>
```

²⁸<https://www.php.net/manual/de/function.echo.php>

Ausgabe mit print()

`print()`²⁹ dient wie `echo` zur Ausgabe von Text. Es handelt sich ebenfalls um ein Sprachkonstrukt, das sich aber wie eine Funktion verhält. Daher wird der auszugebende Text in der Regel auch geklammert (wenngleich die Klammern nicht zwingend notwendig sind):

```
1 print($str);
```

Die drei Beispiele werden mit `print()` folgendermaßen angegeben:

```
1 print("Hallo Welt");
2 print("<h1>Überschrift</h1>");
3 print($text);
```

Da sich `print()` wie eine Funktion verhält, gibt es auch einen Rückgabewert. Dieser ist immer 1.

Wichtige Grundregel bei der Ausgabe

PHP-Skripte können nur Text zurückgeben. Dieser muss im Skript so aufbereitet werden, wie es der Verwendungszweck erfordert.

Dies bedeutet: Soll mit PHP eine Website oder zumindest ein Teil davon (z. B. ein Absatz) dynamisch generiert werden, so genügt es nicht nur, den Text eines Absatzes zurückzugeben, sondern es müssen auch die dazugehörigen HTML-Tags (z. B. `<p></p>`) Teil des zurückgegebenen Texts sein. Selbiges gilt für alle anderen Elemente.

Ein PHP-Skript, das nur aus `echo "Das ist etwas Text."` besteht, kann zwar ohne Probleme im Browser angezeigt werden. Jedoch wird wirklich nur diese eine Zeile Text im Quellcode aufscheinen – weder ein HTML-Grundgerüst, noch HTML-Tags etc. Um eine valide HTML-Seite zu erzeugen, muss entweder das komplette HTML-Grundgerüst mit mehreren `echo`- oder `print()`-Anweisungen ausgegeben werden (sehr umständlich), oder es wird in die PHP-Datei normales HTML-Markup eingefügt, das an den gewünschten Stellen durch PHP-Ausgaben ergänzt wird. Der Beispielcode in Abschnitt 2.3.1 verdeutlicht dies gut.

Der*die Autor*in des PHP-Skripts ist also dafür verantwortlich, dass korrektes HTML-Markup bei der Ausgabe erzeugt wird! Der HTML-Validator³⁰ hilft bei der Überprüfung, ob dies gelungen ist. Selbstverständlich kann dieser aber nur den finalen Output überprüfen, wie er im Browser ankommt. Das PHP-Skript selbst bekommt der Validator nie „zu Gesicht“ (selbst wenn, könnte er mit PHP-Code ohnehin nichts anfangen).

2.3.3 Lexikalische Struktur

Nachdem nun die Ausgabe von Text etwas vorweggenommen wurde, werden im Folgenden die einzelnen Sprachdetails von PHP erläutert. Zunächst die *lexikalische Struktur*. Sie beschreibt die Grundregeln, die für die Sprache gelten.

²⁹<https://www.php.net/manual/de/function.print.php>

³⁰<https://validator.w3.org/>

Groß- und Kleinschreibung

Die *Groß- und Kleinschreibung* in PHP wird gemischt gehandhabt. Bei Namen von Klassen und Funktionen, sowie Sprachkonstrukten und Schlüsselwörtern wie `echo`, `while`, `class` etc. spielt die Schreibweise *keine* Rolle. Die folgenden drei Zeilen sind also äquivalent:

- `echo "Hallo Welt";`
- `ECHO "Hallo Welt";`
- `EcHo "Hallo Welt";`

Bei Variablennamen jedoch spielt die Groß- und Kleinschreibung *immer* eine Rolle. `$name`, `$NAME` und `$NaMe` sind drei unterschiedliche Variablen.

Anweisungen und Strichpunkte

Eine *Anweisung* ist ein Stück Code, der etwas (Sinnvolles) tut. Es kann sich um das Zuweisen eines Werts zu einer Variable oder die Ausführung einer Funktion handeln. Anweisungen in PHP müssen immer mit einem Strichpunkt (;) abgeschlossen werden. Einzige Ausnahme ist die letzte Anweisung, bevor der PHP-Block geschlossen wird. Hier kann das Semikolon entfallen, es sei aber auch hier angeraten, es zu setzen, um mögliche Fehlerquellen zu minimieren. Wie auch in anderen Sprachen, können mehrere Anweisungen in einer Zeile geschrieben werden. Einige Beispiele:

```
1 <?php
2 $a = 1; echo "Hallo Welt!";
3 $name = "Igor";
4 ?>
```

Nach der Anweisung in Zeile 3 könnte der Strichpunkt auch entfallen, da der PHP-Block danach zu Ende ist.

Kommentare

PHP kennt die bereits von Java oder JavaScript bekannten ein- und mehrzeiligen *Kommentare*. Diese werden hinter zwei Schrägstriche (einzeilig) bzw. zwischen Schrägstrich und Stern sowie Stern und Schrägstrich (mehrzeilig) gestellt:

```
1 // Einzeiliges Kommentar
2 /* Ein
3    mehrzeiliges
4    Kommentar */
```

Zusätzlich gibt es noch das einzeilige, sogenannte Unix-Shell-Style bzw. Perl-Kommentar. Das hierbei verwendete Spezialzeichen ist die Raute (#), das Verhalten ist identisch mit dem einzeiligen Kommentar:

```
1 # Ein Unix-Shell-Style Kommentar
```

2.3.4 Datentypen

PHP unterstützt neun primitive *Datentypen*. Vier skalare Typen (Ganzzahlen, Gleitkommazahlen, boolesche Werte und Zeichenketten), drei zusammengesetzte Typen (Arrays, Objekte und Callbacks/Callables), sowie die beiden speziellen Typen `null` und

resource. Zunächst wird auf drei der vier skalaren Typen und null eingegangen, Strings folgen in Abschnitt 2.3.9. Arrays werden in Vorlesung 3 und Objekte in Vorlesung 4 behandelt. Details zu den zwei verbleibenden Typen können dem PHP-Handbuch [5] entnommen werden.

Ganzzahlen

Ganzzahlen oder Integer entsprechen dem Datentyp `long` der Programmiersprache Java. Die Größe ist abhängig von der Plattform und PHP-Version. Mit PHP 8 unter einem 64-bit Betriebssystem sind es üblicherweise 8 Bytes (64 Bits) und können daher einen Wertebereich von $-9.223.372.036.854.775.808$ ($-9,23$ Trillionen bzw. $-9,23 \cdot 10^{18}$) bis $9.223.372.036.854.775.807$ darstellen. Da nirgends genau festgelegt ist, wie viele Bytes eine Ganzzahl in PHP belegen muss, kann dieser Wert auch abweichen.

Ganzzahlen können als Dezimal-, Oktal-, Hexadezimal- oder Binärzahlen angegeben werden. Oktalzahlen werden ab PHP 8.1.0 mit führendem „0o“ angegeben, Hexadezimalzahlen wird „0x“ vorangestellt, bei Binärzahlen kommt „0b“ zum Einsatz:

- 735 (dezimal),
- 0o1337 (oktal),
- 0x2DF (hexadezimal),
- 0b001011011111 (binär).

Seit PHP 7.4 können Ganzzahlen durch den *numeric literal separator* besser lesbar geschrieben werden. Der Underscore (`_`) fungiert hierbei als Trenner für Dezimalstellen und kann bei Dezimalstellen etwa nach jeder dritten Zahl und bei Hexadezimal- und Binärzahlen nach jeder Vierergruppe eingefügt werden. Das Zeichen hat keine Auswirkung auf den Wert und wird vom Interpreter bei der Verarbeitung zuvor entfernt. Als einzige Restriktion gilt, dass der Underscore immer zwischen zwei Zahlen stehen muss. Er darf also nicht am Anfang oder Ende eines Zahlenwert vorkommen bzw. dürfen nicht mehrere Underscores aufeinanderfolgen. Eingesetzt kann er z. B. wie folgt werden:

- 1_000_000 (dezimal),
- 0xCAFE_FOOD (hexadezimal),
- 0b0101_1111 (binär).

Gleitkommazahlen

Gleitkommazahlen in PHP entsprechen dem Datentyp `double` in Java, sind also üblicherweise 8 Bytes (64 Bits) lang. Somit lassen sich Zahlen im Bereich von $-1,7 \cdot 10^{-308}$ bis $1,7 \cdot 10^{308}$ darstellen. Wiederum kann der Bereich vom aktuell verwendeten Betriebssystem abhängen.

Gleitkommazahlen können in zwei Schreibweisen angegeben werden. Mit Dezimalpunkt oder in der Exponentialschreibweise. In letzterer wird ein Wert mit einem „e“, gefolgt vom Exponenten notiert, was $\cdot 10^x$ („Mal zehn hoch x “) bedeutet:

- 3.1415,
- 6.02e7 (entspricht $6,02 \cdot 10^7$ bzw. 60.200.000).

Auch hier kann der *numeric literal separator* eingesetzt werden, um etwa lange Kommastellen besser lesbar zu machen:

- 6.674_083e-11 (entspricht $6,674083 \cdot 10^{-11}$ bzw. 0,00000000006674083).

Boolesche Werte

Die *booleschen Werte* **true** und **false** können in PHP genau wie in anderen Sprachen verwendet werden. Die Groß- und Kleinschreibung spielt dabei keine Rolle (auf php.net wird etwa meist **TRUE** und **FALSE** verwendet, da in PHP konstante Werte per Konvention in Großbuchstaben geschrieben werden).

In PHP ergeben auch andere Werte bei der Umwandlung auf den Typ Boolean den Wert **false**, können also mit logischen Operatoren verglichen werden:

- Der Integer 0,
- die Gleitkommazahlen 0.0 und -0.0,
- ein leerer String ("") und der String "0",
- ein Array ohne Elemente und
- der Typ **null**.

Alle anderen Werte ergeben somit **true**, z. B. 5, 3.5, "Hallo" oder auch [3, 5].

null

Der spezielle Datentyp **null** wird verwendet, um auszusagen, dass eine Variable keinen Wert enthält. Auch hier spielt die Groß- und Kleinschreibung keine Rolle, oft wird daher auch **NULL** verwendet. Eine neu erstellte Variable, der noch kein Wert zugewiesen wurde, hat immer den Wert **null**.

2.3.5 Variablen

Das *Variablenkonzept* von PHP unterscheidet sich in einigen Dingen von anderen Sprachen wie Java oder JavaScript. Verantwortlich dafür sind etwa die implizite Deklaration oder auch das schwache Typkonzept. Auch die Namensgebung ist speziell.

Variablennamen

Die Namen von PHP-Variablen beginnen immer mit einem Dollarzeichen (\$). Danach folgt entweder ein ASCII-Zeichen (Groß- oder Kleinbuchstabe von A–Z) oder ein Unterstrich (_). Alle darauffolgenden Zeichen können entweder wiederum ASCII-Zeichen, Unterstriche oder auch Zahlen von 0–9 sein. Vorsicht: PHP unterstützt ausschließlich ASCII-Zeichen bei der Benennung von Variablen. Umlaute oder andere, seltenere Sonderzeichen sind nicht erlaubt und führen zu Fehlern. Die Groß- und Kleinschreibung spielt, wie bereits in Abschnitt 2.3.3 erwähnt, eine Rolle.

Beispiel für gültige Variablennamen sind etwa:

- \$counter,
- \$nr_of_items,
- \$MaxSpeed oder
- \$_temp.

Folgende Variablennamen sind *nicht* gültig:

- `$current score` (Leerzeichen),
- `$überlauf` (Umlaut),
- `$|dash` (non-ASCII-Zeichen),
- `$3fach` (beginnt mit Zahl).

Die folgenden Variablen heißen zwar gleich, verweisen aber aufgrund der Groß- und Kleinschreibung auf unterschiedliche Werte:

- `$old_value`,
- `$Old_value`,
- `$old_Value` und
- `$OLD_VALUE`.

Deklaration

Variablen werden in PHP implizit deklariert. Dies bedeutet, es gibt keine spezielle Syntax oder kein eigenes Schlüsselwort, um eine Variable anzulegen. Wann immer der Wert für eine Variable gesetzt wird, wird diese erzeugt. Die Zuweisung eines Werts fungiert also als Deklaration.

Folgende Zeilen legen unterschiedliche Variablen an:

```
1 $number = 12;
2 $name = "Igor";
3 $sum = 3 + 5;
```

Dynamische und schwache Typbindung

Wie auch JavaScript, hat PHP eine *dynamische Typbindung*. Dies bedeutet, dass eine Variable beliebige Datentypen enthalten kann. Somit ist folgendes erlaubt:

```
1 $x = 42;
2 ...
3 $x = "fourtytwo";
```

PHP hat des Weiteren auch eine *schwache Typbindung*. Dies äußert sich darin, dass der Typ einer Variable zur Laufzeit von PHP umgewandelt werden kann, wenn dies notwendig erscheint. Dies wird auch implizite Typumwandlung genannt. Sie findet etwa statt, wenn eine nicht-boolesche Variable mit einem logischen Operator verglichen wird. Hierbei kommen die Regeln, die in Abschnitt 2.3.4 erwähnt wurden, zur Anwendung. Im folgenden Code wird die Integer-Variable `$a` implizit auf `true` umgewandelt und die `if`-Bedingung somit betreten:

```
1 $a = 1;
2 if ($a == true) {
3     echo "Typumwandlung in Boolean ist erfolgt!"; // Wird betreten
4 }
```

Ein anderes Beispiel für implizite Typumwandlung ist der `+`-Operator (siehe auch Abschnitt 2.3.6). Er wandelt alle Operanden in numerische Typen um. Werden also ein Integer-Wert und ein String, der einen numerischen Wert enthält, damit addiert, so geschieht die implizite Umwandlung. Das folgende Beispiel illustriert dies:


```
1 $b = "2";  
2 $b += 4;  
3 echo $b; // 6
```

Soll eine Typumwandlung selbst herbeigeführt werden, ist dies wie in Java über sogenanntes Casting möglich. Durch Voranstellen des gewünschten Zieltyps in Klammer wird die Umwandlung (wenn möglich) vorgenommen. Möglich sind die Typen (`bool`), (`int`), (`float`), (`string`), (`array`) und (`object`). Im folgenden Beispiel wird der String-Wert der auf einen Integer-Wert konvertiert und einer neuen Variable zugewiesen:

```
1 $str = "14";  
2 $i = (int)$str; // 14
```

Die implizite Typumwandlung bietet jedoch auch großes Fehlerpotential, da sie leicht übersehen werden kann. Um sich davor zu schützen, können bei Vergleichsoperationen die Operatoren `===` bzw. `!==` zum Einsatz kommen. Sie überprüfen nicht nur auf gleichen Wert, sondern auf gleichen Typ. Auf das Beispiel von oben bezogen würde die folgende `if`-Bedingung somit *nicht* betreten werden, obwohl eine implizite Typumwandlung von 1 auf `true` stattfindet. Der Operator erkennt jedoch, dass die ursprünglichen Typen unterschiedlich waren:

```
1 $a = 1;  
2 if ($a === true) {  
3     echo "Dieser Operator lässt sich nicht überlisten!"; // Wird nicht betreten  
4 }
```

Mit PHP 8 kam es hierbei zu einer wichtigen Änderung: werden Zahlen und nicht-numerische Strings mit dem `==`-Operator (Vorsicht, *nicht* `===`) verglichen, so erfolgt nun eine Umwandlung der Zahl in einen String und diese werden verglichen. Zuvor wurde der String in eine Zahl umgewandelt (das Ergebnis war hierbei immer 0) und diese wurden verglichen.

```
1 if (0 == "foo") {  
2     echo "true"; // Vor PHP 8  
3 } else {  
4     echo "false"; // Ab PHP 8  
5 }
```

Numerische Strings, also etwa `"42"`, werden nach wie vor in Zahlen umgewandelt und dann mit der Zahl verglichen.

2.3.6 Operatoren

Die verschiedenen *Operatoren* in PHP zur Zuweisung oder zum Vergleich von Werten sind denen von C, Java oder auch JavaScript sehr ähnlich. Die große Ausnahme bildet der String-Konkatenationsoperator, der nicht wie üblich das `+`, sondern der `.` ist. Im Folgenden wird auf die verschiedenen Operatoren eingegangen.

Zuweisungsoperatoren

Der `=`-Operator dient der *Zuweisung* von Werten an Variablen. Er kann mit arithmetischen Operatoren kombiniert werden, um eine Operation und Zuweisung in einem Schritt auszuführen. Folgende Kombinationen sind möglich:

- `=`: Zuweisung.
- `+=`: Addition und Zuweisung.
- `-=`: Subtraktion und Zuweisung.
- `*=`: Multiplikation und Zuweisung.
- `/=`: Division und Zuweisung.
- `%=`: Modulo und Zuweisung.
- `.=`: Stringkonkatenation und Zuweisung.

Einige Beispiele dazu:

```
1 $a = 5;
2 $a += 3; // 8
3 $b = "Hallo";
4 $b .= " WET2"; // Hallo WET2
```

Vergleichsoperatoren

Vergleichsoperatoren vergleichen zwei Werte links und rechts des jeweiligen Operators und geben einen booleschen Ausdruck als Resultat zurück. Folgende, meist aus anderen Sprachen bekannte, Operatoren gehören zu dieser Gruppe:

- `==`: Überprüfung auf gleichen Wert.
- `===`: Überprüfung auf gleichen Wert und gleichen Typ.
- `!=`: Überprüfung auf ungleichen Wert.
- `!==`: Überprüfung auf ungleichen Wert und ungleichen Typ.
- `<`: Kleiner als.
- `>`: Größer als.
- `<=`: Kleiner oder gleich als.
- `>=`: Größer oder gleich als.

Vor allem die Operatoren `==` und `===` sind (wie schon in Abschnitt 2.3.5 gezeigt) interessant. Folgendes Beispiel erläutert nochmals den Unterschied:

```
1 $a = 5;
2 $b = 5.0;
3
4 if ($a == $b) {
5     echo "Werte gleich";
6 }
7 if ($a === $b) {
8     echo "Werte und Typ gleich";
9 }
```

`$a` ist ein Integer mit dem Wert 5, `$b` ist eine Gleitkommazahl mit dem Wert 5,0. Die erste `if`-Bedingung überprüft, ob die Werte gleich sind. Diese Bedingung ist erfüllt, also wird der Text ausgegeben. Die zweite Bedingung testet, ob Wert *und* Typ gleich sind. Sie trifft nicht zu, da die erste Variable eine Ganzzahl, die zweite aber eine Gleitkommazahl ist.

Konditionaler Operator

Ebenfalls eine Art Vergleichsoperator ist der *konditionale Operator*. Er besteht aus den Zeichen `?` und `:`. Vor dem Fragezeichen steht ein boolescher Ausdruck, ergibt dieser `true`, so wird der Wert vor dem Doppelpunkt zurückgegeben, ansonsten der danach. Folgendes Beispiel verdeutlicht dies:

```
1 $test = true;
2 echo $test ? "Hallo" : "Kein Hallo"; // Hallo
```

Null Coalescing Operator

Der *Null Coalescing Operator* `??` verknüpft zwei Operanden miteinander und gibt den ersten zurück, wenn dieser existiert (d. h. nicht `null` ist) oder ansonsten den zweiten.

```
1 $name = $vorname ?? "Nobody";
```

Hierbei wird versucht, der Variablen `$name` der Wert von `$vorname` zuzuweisen. Da diese aber nicht existiert, wird der Wert „Nobody“ zugewiesen. Stünde zuvor die Zuweisung `$vorname = "Jane"`;, würde „Jane“ zugewiesen werden, da die Variable dann existent wäre.

Null Coalescing Assignment Operator

Mit dem *Null Coalescing Assignment Operator* kann ähnlich dem Null Coalescing Operator überprüft werden, ob ein Wert nicht existiert (d. h. `null` ist) und nur dann eine Zuweisung machen.

```
1 $hero ??= "Superman";
```

Existiert die Variable `$hero` *nicht*, dann wird der Wert „Superman“ zugewiesen. Gab es zuvor bereits etwa eine Zuweisung `$hero = "Batman"`;, dann bleibt dieser Wert erhalten und „Superman“ wird nicht zugewiesen.

Andere Schreibweisen hierfür sind etwa `$hero = $hero ?? "Superman"`; (mit dem Null Coalescing Operator) oder mit einer `if`-Bedingung:

```
1 if ($hero === null) {
2     $hero = "Superman";
3 }
```

Spaceship Operator

Der in PHP 7 eingeführte *Spaceship Operator* `<=>` ist ein Sonderfall unter den Vergleichsoperatoren. Er vergleicht zwar zwei Ausdrücke, gibt jedoch keinen booleschen Wert zurück, sondern `-1`, wenn der linke Wert kleiner als der rechte ist, `0`, wenn beide Werte gleich sind oder `1`, wenn der linke Wert größer als der rechte ist. Folgendes Beispiel stellt dieses Verhalten dar:

```
1 echo 1 <=> 1; // 0
2 echo 1.5 <=> 2.5; // -1
3 echo "b" <=> "a"; // 1
```

Arithmetische Operatoren

Arithmetische Operatoren repräsentieren die Grundrechnungsarten sowie die Modulo-, Potenz- und Inkrement-/Dekrement-Operatoren. Folgende Operatoren stehen zur Verfügung:

- `+`: Addition (aber *nicht* Strings zusammenhängen!).
- `-`: Subtraktion.
- `*`: Multiplikation.
- `/`: Division.
- `%`: Modulo (Rest der Division).
- `**`: Potenz.
- `++`: Inkrement (post und prä möglich).
- `--`: Dekrement (post und prä möglich).

Dazu einige Beispiele:

```
1 $a = 5 + 3; // 8
2 $b = 5 % 2; // 1
3 $c = ++$a; // 9 (Prä-Inkrement: $a wird auf 9 erhöht und $c zugewiesen)
4 $d = $a++; // 9 (Post-Inkrement: $a wird mit 9 $d zugewiesen und dann auf 10 erhöht)
```

Stringkonkatenation

Der wohl am häufigsten verwechselte Operator in PHP ist der zur *Stringkonkatenation* (Konkatenation = Verkettung von Zeichen oder Zeichenketten). In PHP wird dazu nämlich der Punkt (`.`) und *nicht* wie in den meisten anderen Sprachen das Plus-Zeichen (`+`) verwendet. Diese „Eigenheit“ wurde von der Skriptsprache Perl übernommen. Im folgenden Beispiel werden zwei Zeichenketten zusammengehängt:

```
1 $str = "Hallo " . "Welt";
2 echo $str; // Hallo Welt
```

Durch das schwache Typkonzept von PHP ist bei der Verwechslung von `.` und `+` jedoch Vorsicht geboten. Denn die folgende Anweisung wird vom PHP-Interpreter in PHP-Versionen vor 8.0 ohne Fehlermeldung ausgeführt:

```
1 $str = "Hallo " + "Welt";
2 echo $str; // 0, aber nur für PHP < 8.0
```

Die Ausgabe ist in diesem Fall die Zahl 0, denn PHP nimmt eine implizite Typumwandlung vor und wandelt beide Strings in Integer um. Da weder in „Hallo“ noch in „Welt“ numerische Werte enthalten sind, ist für beide Teile das Ergebnis 0 und $0 + 0$ ergibt schließlich wiederum 0. Ab PHP 8 wird diese Umwandlung nicht mehr durchgeführt und es kommt zu einem schweren Fehler im Skript (*Fatal error: Uncaught TypeError: Unsupported operand types: string + string*).

Auch auf den umgekehrten Fall sei hingewiesen. Gegeben sei folgender Code:

```
1 $first = 100;
2 $second = 50;
3 echo $first . $second; // 10050
4 echo $first + $second; // 150
```

Die beiden Variablen enthalten Integerwerte. Bei der ersten Ausgabe mit `echo` werden diese mit einem Punkt aneinandergehängt. PHP konvertiert aufgrund des Konkatinationsoperators beide Ganzzahlen in Zeichenketten und hängt sie aneinander. Somit wird 10050 ausgegeben. Bei der zweiten Ausgabe werden beide Integerwerte mit dem arithmetischen Operator addiert, somit ist 150 das Ergebnis.

Die Beispiele illustrieren, dass hier durch einfache Verwechslung des Operators völlig unterschiedliche Ergebnisse und somit Fehler entstehen können, die oft nur schwer aufzufinden sind, da die Operationen alle syntaktisch korrekt sind. In PHP 8 wurde aber ein großer Teil dieses Problems, nämlich die (fälschliche) Umwandlung von nicht-numerischen Strings in Zahlen, gelöst.

Logische Operatoren

Zur booleschen Algebra unterstützt PHP die üblichen *logischen Operatoren*:

- `&&`: Logisches UND.
- `||`: Logisches ODER.
- `!`: Logisches NICHT.

Auch hierzu drei Beispiele:

```
1 $a = (4 > 2) && (5 < 3); // false
2 $b = !$a; // true
3 $c = ($a === false) || ($b === false) // true
```

Bitweise Operatoren

PHP unterstützt die üblichen *bitweisen Operatoren* `&` (bitweises UND), `|` (bitweises ODER), `^` (bitweises Exklusiv-ODER), `~` (bitweises NICHT), sowie `<<` (bitweise Verschiebung nach links) und `>>` (bitweise Verschiebung nach rechts). Wie auch schon bei JavaScript sei an dieser Stelle auf die fortführenden Programmierlehrveranstaltungen bzw. die Lehrveranstaltung Data Analysis verwiesen, wo diese Thematik ausführlich behandelt wird.

2.3.7 Kontrollstrukturen

PHP bietet die gängigen *Kontrollstrukturen*, nämlich Verzweigungen und Schleifen, an. Neben der bekannten Schreibweise mit geschwungenen Klammern existiert auch eine Schreibweise mit Doppelpunkten. Diese wird zur Vollständigkeit bei den Verzweigungen demonstriert, gilt aber auch bei den anderen Kontrollstrukturen.

Verzweigungen

Die `if-else`-Bedingung ist, wie auch in anderen Sprachen, die gebräuchliche Art, um Verzweigungen zu realisieren. Die gängige Syntax mit geschwungenen Klammern lautet:

```
1 if (Ausdruck) {
2     Anweisung;
3 } else {
4     Alternative Anweisung;
5 }
```

Wie bereits in der Einleitung zu den Kontrollstrukturen erwähnt, erlaubt PHP auch eine alternative Schreibweise mit Doppelpunkt statt der öffnenden Klammer und abschließendem Statement (hier **endif**) statt der schließenden Klammer. Diese Syntax ist gleichwertig, keinesfalls sollte sie jedoch mit der Klammerschreibweise vermischt eingesetzt werden:

```
1 if (Ausdruck):
2     Anweisung;
3 else:
4     Alternative Anweisung;
5 endif;
```

Wie auch in anderen Sprachen üblich, können im **else**-Zweig wieder neue **if**-Bedingungen begonnen werden. Diese Kaskade kann, wie auch schon bei JavaScript demonstriert, durch eine **switch**-Anweisung ersetzt werden. Folgt in einem **else**-Zweig ein weiteres **if**, soll stattdessen **elseif** verwendet werden. Es wird bevorzugt, da es sich um ein einziges Schlüsselwort handelt.

Es folgen einige verschiedene Beispiele:

```
1 // if-else-Bedingung
2 if ($name !== "") {
3     $username = $name;
4 } else {
5     $username = "John Doe";
6 }
7
8 $n = 1;
9
10 // if-elseif-else-Kaskade
11 if ($n === 1) {
12     echo "Die Zahl ist 1";
13 } elseif ($n === 2) {
14     echo "Die Zahl ist 2";
15 } else {
16     echo "Die Zahl ist weder 1 noch 2";
17 }
18
19 // Switch-Anweisung statt Kaskade
20 switch ($n) {
21     case 1:
22         echo "Die Zahl ist 1";
23         break;
24     case 2:
25         echo "Die Zahl ist 2";
26         break;
27     default:
28         echo "Die Zahl ist weder 1 noch 2";
29         break;
30 }
```

PHP 8: Match Expressions: Statt einer **switch**-Anweisung kann ab PHP 8 auch ein **match**-Ausdruck verwendet werden. Nach dem Schlüsselwort **match** folgt in Klammern der zu evaluierende Ausdruck. Danach folgen in geschweiften Klammern mit Bindestrich getrennte Schlüssel-Wert-Paare, die in der Form **schlüssel => wert** angegeben

werden. Der Schlüssel wird mit dem Ausdruck in der Klammer verglichen, gibt es eine Übereinstimmung, wird der dazugehörige Wert zurückgegeben. Der Vergleich erfolgt dabei intern mit `===` (bei `switch` wird nur `==` verwendet). Die obige `switch`-Anweisung sieht wie folgt als Match Expression aus (`echo` steht hier nur vor `match` um das Ergebnis auszugeben):

```
1 echo match ($n) {  
2   1 => "Die Zahl ist 1",  
3   2 => "Die Zahl ist 2",  
4   default => "Die Zahl ist weder 1 noch 2"  
5 };
```

Schleifen

Auch in PHP finden sich die bekannten Vertreter der *Schleifen* wieder: `while`, `do-while` und `for`.

`while`-Schleifen haben die folgende Syntax:

```
1 while (Ausdruck) {  
2   Anweisung;  
3 }
```

Solange der Ausdruck im Schleifenkopf `true` ergibt, wird die Anweisung in der Schleife ausgeführt. Ist der Ausdruck nie `true`, wird der Rumpf der Schleife auch nie betreten.

Anders bei der `do-while` Schleife. Hier wird der Rumpf auf jeden Fall ein Mal ausgeführt und dann erst überprüft, ob die Anweisung wiederholt werden soll. Die Syntax lautet wie folgt:

```
1 do {  
2   Anweisung;  
3 } while (Ausdruck);
```

Während bei der `while`-Schleife oft „händisch“ eine Zählervariable mit verändert werden muss, um ein Abbruchkriterium zu erhalten, geschieht dies in der `for`-Schleife automatisch. Im Kopf dieser Schleife wird eine Zählervariable initialisiert, auf das Abbruchkriterium überprüft und verändert (z.B. erhöht). Die Syntax lautet hierfür folgendermaßen:

```
1 for (Initialisierung; Test; Veränderung) {  
2   Anweisung;  
3 }
```

Es folgen nun Beispiele für alle drei Schleifentypen. Gerade bei der `for`-Schleife wird auf das `$`-Zeichen bei der Zählervariable im Schleifenkopf vergessen, was zu kryptischen Fehlermeldungen führen kann.

```
1 // while-Schleife  
2 $count = 0;  
3 while ($count < 10) {  
4   // Mach etwas hier...  
5   $count++;  
6 }  
7  
8 // do-while-Schleife  
9 $count = 0;
```

```
10 do {
11     // Mach etwas hier...
12 } while ($count < 10);
13
14 // for-Schleife
15 for ($count = 0; $count < 10; $count++) {
16     // Mach etwas hier...
17 }
```

2.3.8 Advanced Escaping

Dass HTML-Markup und PHP-Code in einer Datei zeilenweise miteinander vermischt werden können, wurde bereits des Öfteren erwähnt. Hier nochmals ein simples Beispiel mit einer Verzweigung:

```
1 <?php
2 if ($wert === "Hallo") {
3     echo "<p>Hallo zurück! :)</p>";
4 } else {
5     echo "<p>Kein Hallo für dich! :(</p>";
6 }
7 ?>
```

Dieses kurze Skript überprüft den Wert einer Variablen und gibt davon abhängig mit `echo` einen Text (als HTML-Absatz ausgezeichnet) zurück. Mit dem sogenannten *Advanced Escaping* kann hier die Verzahnung von HTML- und PHP-Code noch weiter vorangetrieben werden. Denn Inhalte von Blöcken (alles zwischen `{` und `}`) können auch rein in HTML vorliegen. Somit kann statt der `echo`-Aufrufe zunächst der PHP-Block mit `?>` beendet werden. Danach folgt normal der HTML-Code. Wiederum danach wird der PHP-Block vor der schließenden Klammer wieder mit `<?php` begonnen. Dies sieht dann folgendermaßen aus:

```
1 <?php
2 if ($wert === "Hallo") {
3     ?>
4     <p>Hallo zurück! :)</p>
5     <?php
6     } else {
7     ?>
8     <p>Kein Hallo für dich! :(</p>
9     <?php
10    }
11    ?>
```

Diese Syntax macht das Skript in der Regel viel unübersichtlicher, hat aber einen entscheidenden Vorteil bzw. Anwendungsfall: Sollten hier statt einer Zeile Text gleich mehrere ausgegeben werden, können durch Advanced Escaping mehrere `echo`-Aufrufe (und damit verbundenes Verpacken von Text in Anführungszeichen) vermieden werden.

2.3.9 Strings und Stringfunktionen

Strings, auch Zeichenketten genannt, sind ein sehr wichtiger, wenn nicht überhaupt der wichtigste Datentyp in der Webentwicklung. Daten, die Benutzer*innen in Formulare eingeben, kommen als Strings an. Alle Daten, die vom PHP-Programm mittels `echo`

oder `print()` ausgegeben werden, sind ebenfalls Strings. Da PHP mit einigen Besonderheiten in puncto Strings aufwartet, werden diese hier ausführlicher als die restlichen Datentypen (siehe Abschnitt 2.3.4) behandelt.

Strings angeben

Strings können in PHP auf vier unterschiedliche Arten und Weisen angegeben werden:

- In einfachen Anführungszeichen (engl. „single-quoted string“),
- in doppelten Anführungszeichen (engl. „double-quoted string“),
- in der Heredoc-Syntax und
- in der Nowdoc-Syntax.

Strings in einfachen Anführungszeichen: Die einfachste Art, einen String anzugeben, ist, ihn in einfache Anführungszeichen (`' '`) zu setzen. Alle Zeichen innerhalb des Strings werden 1:1 so wiedergegeben, wie sie spezifiziert wurden, es gibt nur zwei Ausnahmen: Wird das einfache Hochkomma innerhalb des Strings benötigt, muss es mit einem Backslash „escaped“ werden (`\'`), wird der Backslash selbst benötigt, muss er mit zwei Backslashes angegeben werden (`\\`). Spezialzeichen wie Zeilenumbrüche (`\n`), Tabulatoren (`\t`) oder auch Variablen (z.B. `$text`) werden nicht ersetzt, sondern genauso ausgegeben. Die folgenden Beispiele illustrieren dies:

```
1 echo 'Hallo'; // Hallo
2 echo 'Hallo\nWelt'; // Hallo\nWelt
3 echo 'Wichtig: $text'; // Wichtig: $text
4 echo 'PHP ist \'toll\''; // PHP ist 'toll'
```

Strings in doppelten Anführungszeichen: Werden Strings in doppelten Anführungszeichen angelegt, kommt es zur Interpretation von Variablen (sog. Variableninterpolation) und ebenfalls zur Ersetzung von diversen Spezial- bzw. Escapezeichen. Werden also Variablennamen (z.B. `$text`) oder Spezialzeichen wie Zeilenumbrüche (`\n`) oder Tabulatoren (`\t`) verwendet, so werden diese durch ihre dahinter liegenden Bedeutungen ersetzt. Doppelte Anführungszeichen müssen durch einen Backslash escaped werden (`\"`), um nicht vorzeitig den String zu beenden. Die folgenden Beispiele sollen dies darstellen:

```
1 echo "Hallo"; // Hallo
2 echo "Hallo\nWelt"; // Hallo
3                      // Welt
4 $text = "Variable wurde interpoliert!";
5 echo "Wichtig: $text"; // Wichtig: Variable wurde interpoliert!
6 echo "PHP ist \"toll\""; // PHP ist "toll"
```

Um dem PHP-Parser exakt mitzuteilen, wo der Bezeichner der Variable endet, kann diese in geschwungene Klammern gesetzt werden (curly syntax). Das ist etwa dann nötig, wenn nach dem Variablennamen danach kein Leerzeichen kommen soll:

```
1 $count = 3;
2 echo "Dauer: {$count}min"; // Dauer: 3min
```

Ohne die Klammern würde PHP versuchen auf eine Variable `$countmin` zuzugreifen, die es nicht gibt. Diese Syntax ist auch beim Zugriff auf zweidimensionale Arrays oder Objekteigenschaften (siehe Vorlesungen 3 und 4) nötig.

Früher galt die Faustregel, dass einfache Anführungszeichen schneller seien, da der Parser nicht nach Variablen suchen muss. Bei modernen PHP-Versionen (OPcache) ist dieser Unterschied jedoch vernachlässigbar. Die Wahl sollte daher primär nach der Lesbarkeit (Interpolation nötig oder nicht?) oder einfach nach persönlicher Präferenz getroffen werden.

Heredoc-Syntax: Die Heredoc-Syntax (kurz für Here Documents) wurde mit PHP4 eingeführt, um zu ermöglichen, längere Texte mit Zeilenumbrüchen in Strings zu speichern, ohne diese mittels `\n` erzeugen zu müssen.

Um einen Heredoc-String zu beginnen, werden drei öffnende spitze Klammern gefolgt von einem möglichst eindeutigen Identifikator angegeben. Danach muss eine neue Zeile für den Text begonnen werden. Abgeschlossen wird der Heredoc-Block vom Identifikator, gefolgt von einem Strichpunkt. Der abschließende Identifikator kann eingerückt werden. In diesem Fall werden genau diese Anzahl Leerzeichen auch aus jeder Zeile des Heredoc entfernt (bessere Lesbarkeit im Code).

Der Heredoc-Block verhält sich identisch mit einem String in doppelten Anführungszeichen. Variablen und Spezialzeichen werden also ersetzt. Einzig doppelte und einfache Anführungszeichen können im Heredoc normal (d. h. ohne Backslash) verwendet werden.

Hier ein Beispiel für die Heredoc-Syntax. Als Identifikator wird „EOT“ verwendet. Es könnte aber genauso „ENDE“ oder jede andere Zeichenkette gewählt werden. Diese sollte nur nicht genauso im Text vorkommen.

```
1 $name = "John Doe";
2 $hdText = <<<EOT
3   <p class="bold">Dies ist
4   ein Text von $name, der
5   genauso durch die
6   Zeilenumbrüche im String
7   gespeichert wird.</p>
8   EOT;
```

Im Text wird `$name` durch „John Doe“ ersetzt und genauso mit den Zeilenumbrüchen im String gespeichert.

Nowdoc-Syntax: Die Nowdoc-Syntax existiert als Gegenstück zu Heredoc. Sie verhält sich zur Heredoc-Syntax so, wie sich ein String unter einfachen Hochkommas zu einem unter doppelten Hochkommas verhält, d. h. Variablen und Spezialzeichen werden nicht ersetzt.

Der Nowdoc-Bereich wird ebenfalls mit drei spitzen Klammern und einem Identifikator begonnen, allerdings steht der Identifikator unter einfachen Anführungszeichen. Beendet wird der Block wieder über die Angabe des Identifikators.

Das folgende Beispiel ist identisch zum vorigen, jedoch als Nowdoc-Block:

```
1 $name = "John Doe";
2 $hdText = <<<'EOT'
3   <p class="bold">Dies ist
4   ein Text von $name,
5   der genauso durch die
6   Zeilenumbrüche im String
7   gespeichert wird.</p>
8   EOT;
```

Die Variable `$name` wird im Text nicht ersetzt, im String steht daher „von `$name`“ und nicht „von John Doe“.

Stringfunktionen

Für die Arbeit mit Zeichenketten bietet PHP eine umfangreiche Sammlung an *Stringfunktionen*, von denen in der Folge einige wichtige exemplarisch vorgestellt werden. Eine komplette Übersicht findet sich in der PHP-Dokumentation³¹. Diese Stringfunktionen stammen jedoch aus frühesten PHP-Tagen und gehen daher von der (naiven) Annahme aus, dass jedes Zeichen ein 8-bit (d. h. 1 Byte) Zeichen ist. Dies ist für alle Zeichen des ASCII-Zeichensatzes der Fall, sobald jedoch Sonderzeichen ins Spiel kommen, wie etwa deutsche Umlaute oder auch so gut wie alle Unicode-Zeichen, führt dies zu Problemen, da diese Zeichen mit mehr als einem Byte repräsentiert werden. Daher wird bei den Stringfunktionen – wo immer möglich – auf die Funktionen der **mbstring**-Erweiterung zurückgegriffen. Diese beinhalten die meisten der nativen PHP-Stringfunktionen, nehmen jedoch auf Zeichen, die mehrere Bytes (Multibytes) benötigen, Rücksicht. Das erste Beispiel zur Länge einer Zeichenkette illustriert die Wichtigkeit dieser Multibyte-Funktionen. Eine volle Auflistung findet sich ebenfalls im PHP-Handbuch³².

Die hier aufgezählten Funktionen werden inklusive ihrer Parameter angegeben. Manche dieser Parameter stehen in eckigen Klammern, dies bedeutet, dass sie optional sind, also nicht angegeben werden müssen.

Länge einer Zeichenkette: Die Funktion `strlen($string)`³³ ermittelt die Länge eines Strings. Zurückgegeben wird eine positive Ganzzahl oder 0, wenn der String leer ist. Das folgende Beispiel stellt dies dar:

```
1 $str = "Hallo";
2 echo strlen($str); // 5
```

In diesem Beispiel kommen nur ASCII-Zeichen mit einem Byte Länge vor, weshalb die Funktion das erwartete Ergebnis liefert. Sobald jedoch Multibyte-Zeichen (wie etwa ein Umlaut) vorkommen, wird die Funktion `mb_strlen($string [, $encoding])`³⁴ benötigt. Das Beispiel demonstriert den Unterschied:

```
1 $str = "Hällo";
2 echo strlen($str); // 6
3 echo mb_strlen($str); // 5
```

Während `strlen()` die Bytes zählt um zum (falschen) Ergebnis 6 kommt, berücksichtigt `mb_strlen()`, dass das Sonderzeichen ein Multibyte-Zeichen ist und gibt die Stringlänge korrekt mit 5 an.

Zugriff auf Einzelzeichen: Der Zugriff auf einzelne Zeichen eines Strings geschieht wie der Zugriff auf Elemente eines Arrays mit Hilfe der eckigen Klammern (`[]`). Das erste Zeichen hat Position 0, das letzte befindet sich an der Position *AnzahlZeichen* – 1. Folgendes Beispiel illustriert dies:

³¹<https://www.php.net/manual/de/ref.strings.php>

³²<https://www.php.net/manual/de/book.mbstring.php>

³³<https://www.php.net/manual/de/function strlen.php>

³⁴<https://www.php.net/manual/de/function.mb-strlen.php>

```
1 $str = "Hallo Welt";
2 echo $str[0]; // H
3 echo $str[3]; // l
```

Extrahieren von Substrings: Soll auf Ausschnitte einer Zeichenkette (sogenannte Substrings) zugegriffen werden, bietet sich dafür die Verwendung der Multibyte-Funktion `mb_substr($string, $start [, $length [, $encoding]])`³⁵ an. `$string` ist dabei die Zeichenkette, aus der ein Teil entnommen werden soll. `$start` gibt den Start-Offset an, also ab welchem Zeichen die Extraktion stattfinden soll. Optional kann mit `$length` angegeben werden, wie viele Zeichen entnommen werden sollen. Entfällt der Wert, wird bis zum Ende des Strings gearbeitet. Beide Werte können auch negativ sein. Bei `$start` wird vom dann Ende des Strings weg zu zählen begonnen, ein negatives `$length` gibt an, wie viele Zeichen vom Ende des Strings abgeschnitten werden sollen. Folgende Beispiele verdeutlichen die Funktion:

```
1 $str = "Hallo Welt";
2 echo mb_substr($str, 2, 3); // llo
3 echo mb_substr($str, -2); // lt
4 echo mb_substr($str, -3, 2); // el
5 echo mb_substr($str, 2, -3); // llo W
```

Entfernen unnötiger Leerzeichen: Oft befinden sich am Anfang oder Ende eines Strings Leerzeichen, die durch Zufall (schlampiges Copy & Paste etc.) hinzugekommen sind. Diese Leerzeichen können mitunter problematisch werden, wenn dadurch ein Benutzer*innenname etwa als „JohnDoe_“ abgespeichert wird.

Die Funktion `trim($string [, $charlist])`³⁶ schafft hier Abhilfe. Der Parameter `$string` ist die zu überprüfende Zeichenkette. Ohne das zweite Argument werden Leerzeichen, Tabulatoren, Zeilenumbrüche und noch eine Reihe von Spezialzeichen entfernt. Wird zusätzlich `$charlist` als zweiter Parameter angegeben, werden die in diesem String enthaltenen Zeichen ebenfalls entfernt. Zwei Beispiele:

```
1 $str = trim(" Hallo Welt "); // Hallo Welt
2 $more = trim("!-Hallo Welta", "a-!"); // Hallo Welt
```

Anmerkung: Es gibt keine `mb_trim()` Funktion, da die gängigen Whitespace-Zeichen alle nur ein Byte lang sind und daher auch von einer Zeichenkette mit Multibyte-Zeichen korrekt entfernt werden. Wird allerdings eine Trim-Funktion benötigt, die explizit Multibyte-Zeichen entfernt (zweiter Parameter bei `trim()`), so muss diese selbst geschrieben werden.

Vergleichen von Strings: Die Funktion `strcmp($str1, $str2)`³⁷ überprüft zwei Strings auf Gleichheit. Sind die Werte gleich, wird 0 zurück gegeben, ist `$str1` (lexikografisch) kleiner als `$str2` wird eine negative Zahl zurückgegeben. Im umgekehrten Fall ist eine positive Zahl das Ergebnis des Vergleichs. Die Größe der Zahl richtet sich dabei nach dem Abstand der Zeichen in der ASCII-Tabelle. Wird „A“ mit „D“ vergli-

³⁵<https://www.php.net/manual/de/function.mb-substr.php>

³⁶<https://www.php.net/manual/de/function.trim.php>

³⁷<https://www.php.net/manual/de/function strcmp.php>

chen ist das Ergebnis -3 , denn A ist 3 Stellen vor D in der ASCII-Tabelle positioniert. Groß- und Kleinschreibung wird beachtet. Soll diese ignoriert werden, muss die Funktion `strcasecmp($str1, $str2)`³⁸ verwendet werden. Achtung: Es gibt es kein `mb_strcmp()`, da der Vergleich etwa von Unicode-Zeichen oder auch Zeichen unterschiedlicher Kodierung ungleich komplexer ist und in der Regel Spezial-Implementierungen benötigt (z. B. mit einem sogenannten Collator-Objekt).

Strings können auch mit den Vergleichsoperatoren `==`, `===`, `<`, `>`, `<=`, `>=`, `>==`, `>==` und `<=>` durchgeführt werden. `strcmp()` ist hier vorzuziehen, da die Funktion beim Vergleichen von Strings und Zahlen auf String konvertiert, während die Operatoren vor PHP 8 noch auf Integer konvertieren. Dazu einige Beispiele:

```

1 $value1 = 3;
2 $value2 = "3";
3 $name1 = "Fred";
4 $name2 = "Wilma";
5
6 /* Vergleich zweier Integer mit strcmp */
7 echo strcmp($value1, $value2); // 0
8 /* Vergleich auf Wert (==) */
9 if ($value1 == $value2) {
10     echo "Wert gleich"; // wird ausgegeben
11 }
12 /* Vergleich auf Wert und Typ (===) */
13 if ($value1 === $value2) {
14     echo "Wert und Typ gleich"; // wird nicht ausgegeben
15 }
16 /* Lexikografischer Vergleich zweier Strings */
17 if ($name1 < $name2) {
18     echo "Fred kommt vor Wilma"; // wird ausgegeben
19 }
20 /* Lexikografischer Vergleich von String mit Integer */
21 if ($name1 < $value1) {
22     echo "Fred < 3"; // // Vor PHP 8 ausgegeben (aber falsch!), mit PHP 8 nicht ausgegeben
23 }
24 /* Vergleich mit strcmp von String mit Integer */
25 echo strcmp($name1, $value1); // 19 (F ist in der ASCII-Tabelle 19 Stellen hinter 3)

```

Das erste Beispiel (Zeile 6) vergleicht eine Zahl und einen String mittels `strcmp()`. Der Integer wird automatisch zu einem String konvertiert und der Vergleich ergibt 0 – somit sind die Strings gleich.

In Beispiel zwei (Zeile 8) werden die Werte mittels `==` auf Gleichheit überprüft. Da die Werte gleich sind, wird die `if`-Bedingung betreten und der Text ausgegeben.

Im dritten Beispiel (Zeile 12) wird mit dem `===`-Operator auf Wert- und Typgleichheit überprüft. Da ein Integer und ein String verglichen werden, ergibt der Vergleich `false` und die Bedingung wird nicht betreten.

In Beispiel vier (Zeile 16) werden zwei Strings lexikografisch mit dem `<` Operator verglichen. Da das „F“ von Fred vor dem „W“ von Wilma kommt, ergibt der Vergleich `true` und der Text wird ausgegeben.

Im fünften Beispiel (Zeile 20) kommt es hingegen bei PHP-Versionen kleiner 8.0 zu Problemen. Es werden ein String und ein Integer verglichen, der String ist jedoch nicht numerisch. Er wird automatisch von PHP auf einen Integer mit dem Wert 0

³⁸ <http://php.net/manual/de/function.strcasecmp.php>

umgewandelt. 0 ist kleiner als 3, somit wird Fred kleiner als 3 angesehen, obwohl die Buchstaben im lexikografischen Vergleich *nach* den Zahlen kommen. Somit passiert hier ein Fehler. Ab PHP 8 wird dieses Verhalten korrigiert. Die Zahl wird hierbei zu einem String konvertiert, diese ist kleiner (kommt vor den Buchstaben), daher wird nichts ausgegeben.

In Beispiel sechs (Zeile 24) wird der selbe Vergleich mit `strcmp()` durchgeführt. Nun wird die Zahl 3 in einen String konvertiert und korrekt verglichen. Das Ergebnis ist 19, denn F ist 19 Stellen hinter 3 in der ASCII-Tabelle zu finden.

Suchen von Substrings („Needle in a Haystack“): Mit Hilfe der Multibyte-Funktion `mb_strpos($haystack, $needle [, int $offset = 0 [, $encoding]])`³⁹ kann das *erste* Vorkommen eines Substrings in einem String überprüft werden. `$haystack` ist der String in dem gesucht wird, `$needle` ist der zu suchende Substring. `$offset` ist optional und gibt an, ab welcher Position gesucht werden soll. Dazu zwei Beispiele:

```
1 $text = "testen von strings";
2 $pos1 = mb_strpos($text, "st"); // 2
3 $pos2 = mb_strpos($text, "st", 5); // 11
```

Mit `mb_strrpos($haystack, $needle [, int $offset = 0 [, $encoding]])`⁴⁰ wird nach demselben Schema das *letzte* Vorkommen eines Substrings zurückgegeben:

```
1 $text = "testen von strings";
2 $pos = mb_strrpos($text, "st"); // 11
```

Um nicht den Index, sondern stattdessen den verbleibenden Teil von `$haystack` ab dem ersten Vorkommen von `$needle` zurückzugeben, gibt es die Multibyte-Funktion `mb_strstr($haystack, $needle [, $before_needle = false [, $enc]])`⁴¹. Groß- und Kleinschreibung werden dabei beachtet. Anders handhabt dies die Multibyte-Funktion `mb_stristr($haystack, $needle [, $before_needle = false [, $enc]])`⁴². Ist der dritte, optionale Parameter auf `true` gesetzt, wird der Teil *vor* dem Vorkommen zurückgegeben. Zwei Beispiele dazu:

```
1 $str1 = mb_strstr("Testen von Strings", "St"); // Strings
2 $str2 = mb_stristr("Testen von Strings", "St"); // sten von Strings
```

Umwandeln in Groß- und Kleinbuchstaben: Durch Verwendung der Multibyte-Funktion `mb_convert_case($string, $mode [, $encoding])`⁴³ kann ein String in drei Varianten, abhängig vom Wert des Parameters `$mode` umgewandelt werden. Dieser kann eine von drei Konstanten annehmen:

- `MB_CASE_UPPER`: Wandelt alle Buchstaben in Großbuchstaben um.
- `MB_CASE_LOWER`: Wandelt alle Buchstaben in Kleinbuchstaben um.
- `MB_CASE_TITLE`: Wandelt jeden ersten Buchstaben eines Worts in Großbuchstaben um.

³⁹<https://www.php.net/manual/de/function.mb-strpos.php>

⁴⁰<https://www.php.net/manual/de/function.mb-strrpos.php>

⁴¹<https://www.php.net/manual/de/function.mb-strstr.php>

⁴²<https://www.php.net/manual/de/function.mb-stristr.php>

⁴³<https://www.php.net/manual/de/function.mb-convert-case.php>

Dazu jeweils ein Beispiel:

```
1 $text = "Hallo kleine Welt";  
2 echo mb_convert_case($text, MB_CASE_UPPER); // HALLO KLEINE WELT  
3 echo mb_convert_case($text, MB_CASE_LOWER); // hallo kleine welt  
4 echo mb_convert_case($text, MB_CASE_TITLE); // Hallo Kleine Welt
```

2.4 Weiterführende Literatur

Das Einführungswerk in PHP ist sicherlich [4], da es in seinen ersten drei Auflagen vom Erfinder der Sprache selbst mitverfasst wurde. Das Buch bietet eine komplette Einführung von Grund auf und geht auch auf fortgeschrittenere Themen ein. Seit März 2020 ist die vierte Auflage verfügbar, die alle Features bis inklusive PHP 7.4 enthält und somit wieder einigermaßen aktuell ist. PHP 8 wird allerdings nicht abgedeckt, daher ist es mittlerweile als veraltet zu betrachten, bis eine neue Auflage erscheint.

Eine etwas knappere PHP-Einführung, dafür aber eine ausführliche Einführung in die Welt der MySQL-Datenbanken bietet [6]. Ein derartiges Buch erscheint bei jedem großen PHP- oder MySQL-Release und bietet so immer die letzten Neuerungen, so auch bereits für PHP 8. Inhaltlich richtet es sich zwar durchaus an Anfänger, setzt aber schnell einige Kenntnisse voraus. Wer selbiges in Englisch sucht, wird bei [3] fündig. Dieses Buch beinhaltet auch noch HTML5-, CSS- und JavaScript-Grundlagen und in der neuesten Auflage auch bereits PHP 8.

In dieselbe Kerbe wie die vorigen beiden Empfehlungen schlägt auch [8] – auch hier wird ein Einstieg in PHP 8 und MySQL angeboten, wenngleich das Niveau etwas höher angesetzt ist und sich auch an fortgeschrittene Entwickler*innen richtet.

Lösungen für konkrete PHP-Problemstellungen bietet [2]. Im Buch werden viele alltägliche Probleme, die es beim Programmieren zu lösen gilt heraus, herausgegriffen und effiziente Lösungen inklusive Erklärung präsentiert. Das entsprechende Wissen in PHP wird jedoch bereits vorausgesetzt.

Wer ein Online-Nachschlagewerk sucht, um PHP von Grund auf „richtig“ zu lernen, dem sei das Projekt *PHP: The Right Way* [1] ans Herz gelegt.

Schließlich sei wie beim Erlernen einer jeden Skript- oder Programmiersprache auf die eigene Dokumentation [5] verwiesen. Diese enthält sowohl eine Referenz aller Funktionen und Konstrukte, aber auch Informationen über grundlegende Konzepte bis hin zur Installation.

Quellenverzeichnis

- [1] Josh Lockhart und Phil Sturgeon. *PHP: The Right Way*. 22. Jan. 2026. URL: <https://phptherightway.com/> (besucht am 14.04.2026) (siehe S. 2-30).
- [2] Eric A. Mann. *PHP Cookbook. Modern Code Solutions for Professional Developers*. Sebastopol, USA: O'Reilly, 20. Juni 2023 (siehe S. 2-30).
- [3] Robin Nixon. *Learning PHP, MySQL & JavaScript. A Step-by-Step Guide to Creating Dynamic Websites*. 7. Aufl. Sebastopol, USA: O'Reilly, 18. Feb. 2025 (siehe S. 2-30).
- [4] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 2-30).
- [5] The PHP Documentation Group. *PHP-Handbuch*. 14. Apr. 2026. URL: <https://www.php.net/manual/de/> (siehe S. 2-13, 2-30).
- [6] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 2-30).
- [7] W3Techs. *Usage statistics of server-side programming languages for websites*. 13. Apr. 2026. URL: https://w3techs.com/technologies/overview/programming_language (besucht am 14.04.2026) (siehe S. 2-5).
- [8] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 2-30).